CSC738 Advanced Machine Learning · Topic 3 · Week 5

Supervised Learning II: Regression, Forecasting and Challenges

Regression models, time-series forecasting, evaluation and common pitfalls

Objectives

- Explain how Linear, Ridge, Lasso, KNN, tree-based and SVR models predict continuous values
- Turn a time series into a supervised learning problem and forecast it
- Evaluate regression and forecasting models with MAE, RMSE, R^2 and MAPE
- Analyse results with baselines, residual plots and error breakdowns
- Recognise and handle overfitting, imbalanced data and data leakage

Classification vs regression

	Classification (Week 4)	Regression (this week)
Output	A category: pass/fail, disease type	A number: salary, weight, dengue cases
Target data	Discrete labels	Continuous values
Example question	Will this student pass?	What mark will this student get?
Algorithms	Logistic Regression, SVM, Random Forest classifier	Linear Regression, SVR, Random Forest regressor
Metrics	Accuracy, precision, recall, F1	MAE, RMSE, R^2, MAPE
Baseline	Predict the majority class	Predict the mean (or the last value in time)

Part 1

Regression models

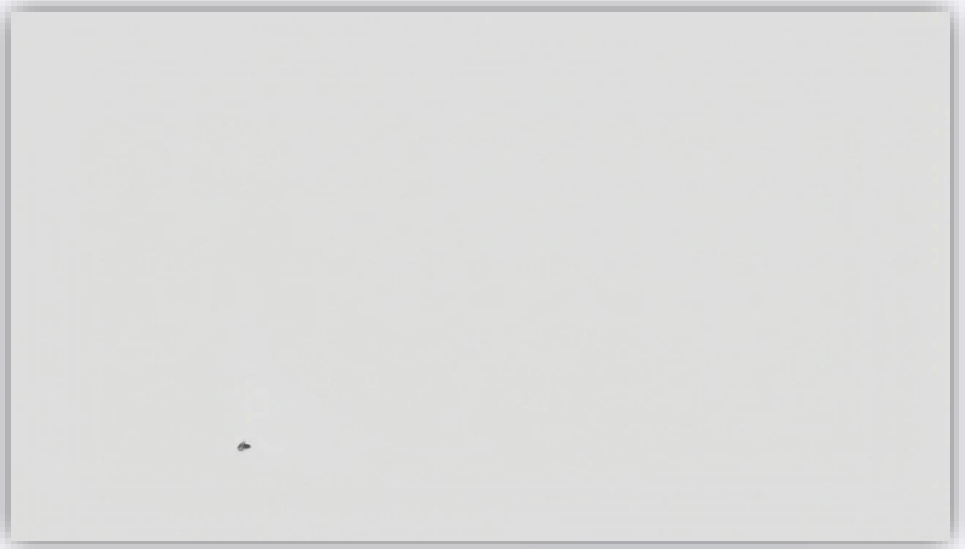
Linear, regularised and non-linear regressors

Linear Regression

Fits a straight line (or plane) through the data:

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \ldots + b_nx_n$$

- Chooses the weights that minimise the sum of squared errors (least squares)
- Orange lines are the residuals: actual minus predicted
- Each weight b shows how much ŷ changes when that feature rises by one unit
- Multiple linear regression = more than one feature





Polynomial features and regularisation

Polynomial regression

Add x^2, x^3 … as new features so a linear model can fit curves. A high degree fits the training data closely but can overfit.

Ridge (L2)

Adds a penalty on large weights; shrinks all of them. Good when features are correlated.

Lasso (L1)

Penalty that can push weights to exactly zero, so it also selects features.

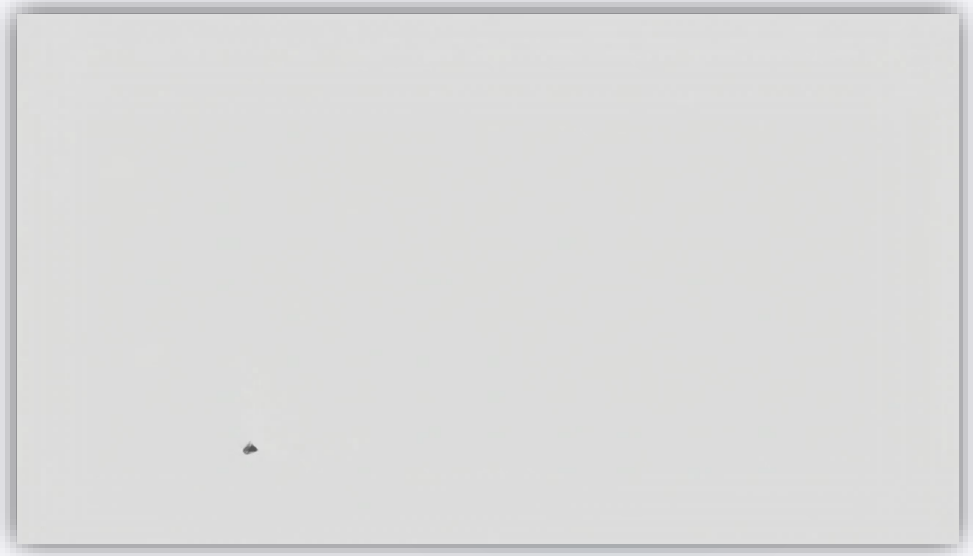
Hyperparameter alpha	Effect
Small alpha	Close to plain Linear Regression: may overfit
Large alpha	Strong shrinking: simpler model, may underfit

Scale the features first and tune alpha with cross-validation.

Support Vector Regression (SVR)

Fits a function with a **tube** of width ε around it:

- Errors inside the tube are ignored
- Only points outside the tube (orange) are penalised, controlled by C
- Kernels (linear, polynomial, RBF) allow curved fits, as in SVM
- Needs scaled features; slow on large datasets



Hyperparameter	Typical values
C	0.1–100
epsilon	0.01–1 (i units)
gamma (RBF)	0.001–1

How strongly errors outside the tube are penalised

Half-width of the tube, in the same units as the target (e.g. ε = 0.5 means errors under 0.5 cases

Flexibility of the RBF curve



tube width = 2ε

Other regressors

Model	How it predicts a number	Scaling	Key hyperparameters
KNN regressor	Average target of the k nearest neighbours	Yes	n_neighbors, weights
Decision Tree regressor	Mean target of the leaf the sample falls into	No	max_depth, min_samples_leaf
Random Forest regressor	Average of all trees' predictions	No	n_estimators, max_depth, max_features
Gradient Boosting / XGBoost	Each tree fits the remaining errors; predictions are added up	No	learning_rate, n_estimators, max_depth

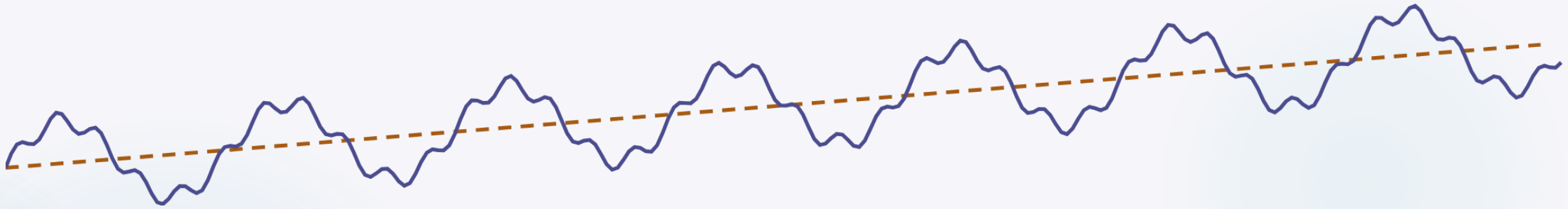
Tree-based regressors cannot predict beyond the range of the training targets, which matters for trends in time series.

Part 2

Time-series forecasting

Order matters: trend, seasonality and time-based validation

What makes time series different



Trend

Long-term rise or fall (dashed orange), e.g. growing electricity use.

Seasonality

Repeating pattern: daily, weekly, yearly, e.g. dengue peaks in rainy months.

Order matters

Rows are not independent: never shufflemand use only the past to predict the future.

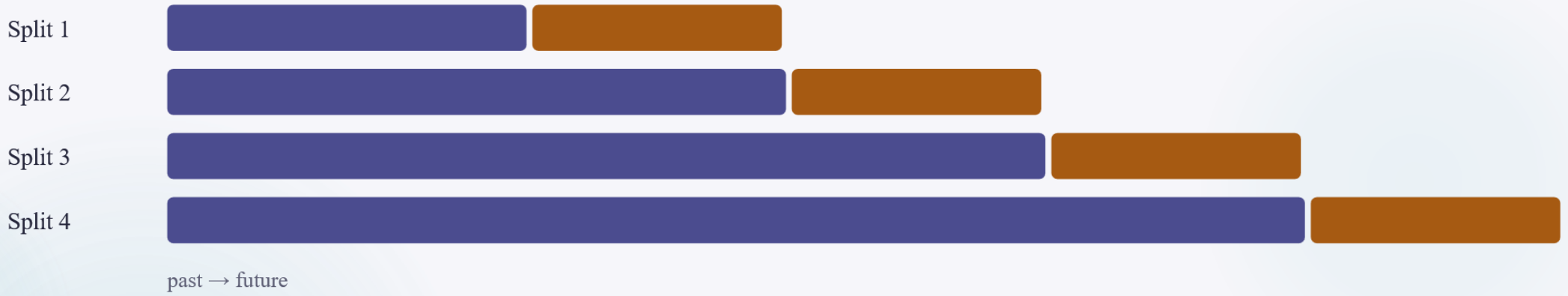
Turning a series into a supervised problem

Each row predicts the value at time t from information available before t.

Week	Cases (target)	lag_1	lag_52	roll_mean_4	rainfall_lag_2
t − 1	120	110	95	104	210 mm
t	?	120	102	111	185 mm

- **Lags:** last week, same week last year
- **Rolling statistics:** computed with shift(1) so they end at t − 1
- **External features:** e.g. rainfall, but only values already known when forecasting

Validating a forecast: split by time



Train (purple), then validate (orange)

Each split trains on the past and validates on the block right after it. In scikit-learn: TimeSeriesSplit.

Final test

Hold out the most recent period, e.g. the last 20% of weeks, and test on it once.

Forecasting baselines

Baseline	Prediction for time t	Good when
Naive	The last value (t − 1)	The series changes slowly
Seasonal naive	The value one season ago (e.g. t − 52 weeks)	Strong seasonality
Moving average	Mean of the last k values	Noisy series without a trend
Mean	Mean of the training target	Non-time-series regression

A machine learning model is only useful if it clearly beats these. Report the baseline in every result table.

Part 3

Evaluation and result analysis

Metrics, a worked example and residual plots

Regression metrics

Metric	Formula (in words)	Units	Read it as
MAE	Mean of $\lvert$actual − predicted$\rvert$	Target units	Typical size of an error
MSE	Mean of (actual − predicted)^2	Units squared	Punishes large errors; hard to read
RMSE	$\sqrt{\text{MSE}}$	Target units	Like MAE, but large errors count more
R^2	1 − SSE / SST	None	Share of the target's variance explained; 1 is perfect, can be negative
MAPE	Mean of $\lvert$error$\rvert$ / $\lvert$actual$\rvert$ × 100%	Percent	Easy for managers; fails when actual values are near 0

Report at least two: one in target units (MAE or RMSE) and R^2 or MAPE.

Worked example: five predictions

Actual	Predicted	Error	Error²
10	11	1	1
12	11	−1	1
15	16	1	1
18	17	−1	1
20	22	2	4

MAE = (1+1+1+1+2) / 5 = **1.2**

MSE = 8 / 5 = 1.6 → RMSE = **1.26**

Mean actual = 15; SST = 68; SSE = 8

R^2 = 1 − 8/68 = **0.88**

MAPE = **8.1%**

RMSE > MAE because the one larger error (2) counts more when squared.

Result analysis: look at the residuals

Good: random scatter around 0



Problem: a pattern (curve)



- A pattern means the model is missing something: a curve, a feature or a seasonal effect
- Also plot actual vs predicted over time, and compare errors by group (e.g. by district or by season)

Part 4

Challenges

Overfitting, imbalanced data and data leakage

Challenge 1: overfitting and underfitting

	Underfitting	Good fit	Overfitting
Training error	High	Low	Very low
Validation error	High	Low, close to training	High
Regression example	Straight line through curved data	Degree-2 curve	Degree-15 polynomial through every point
Fix	More features, more flexible model	Keep it	Regularise (Ridge, Lasso), limit depth, more data, simpler model

Compare training and cross-validated errors, and plot learning curves to see which side you are on.

Challenge 2: imbalanced data

The problem

One class is rare (e.g. 5% fraud, 10% at-risk students). A model can score high accuracy while missing almost every rare case.

In regression, the same happens with rare extreme values, e.g. outbreak weeks with very high dengue cases.

What to do

- Report recall, F1 or PR-AUC, not accuracy alone
- Use class_weight="balanced"
- Resample (e.g. SMOTE) on training folds only
- Move the decision threshold
- Regression: log-transform a skewed target; check errors on the extreme values separately

Challenge 3: data leakage

Type	Example	Fix
Preprocessing leakage	Scaling or imputing with statistics from the whole dataset	Pipeline: fit on training data only
Target leakage	Using "days in hospital" to predict whether a patient will be admitted	Use only features known at prediction time
Time leakage	Shuffling a time series; a rolling mean that includes today	Split by time; shift(1) before rolling
Duplicate leakage	The same record in both training and test sets	Remove duplicates before splitting

Warning sign: results that look too good to be true usually are.

Python: forecasting workflow

```
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

split = int(len(df) * 0.8) # past = train, future = test
X_tr, X_te = X.iloc[:split], X.iloc[split:]
y_tr, y_te = y.iloc[:split], y.iloc[split:]

naive = X_te["lag_1"] # baseline: last week's value
search = GridSearchCV(RandomForestRegressor(random_state=0),
                      {"n_estimators": [200, 500], "max_depth": [None, 10]},
                      cv=TimeSeriesSplit(5),
scoring="neg_mean_absolute_error").fit(X_tr, y_tr)
pred = search.predict(X_te)

for name, p in [("Naive", naive), ("Random Forest", pred)]:
    print(name, mean_absolute_error(y_te, p), mean_squared_error(y_te, p) **
0.5, r2_score(y_te, p))
```

Case study: forecasting weekly dengue cases

Kept from the original deck and extended: predict next week's dengue cases for a district in Selangor.

Step	Decision
Features	Lags of cases (1, 2, 52 weeks), 4-week rolling mean, lagged rainfall and temperature, week of year
Baselines	Seasonal naive (same week last year) and naive (last week)
Models	Linear Regression, Random Forest, Gradient Boosting; tuned with TimeSeriesSplit
Evaluation	MAE and RMSE on the most recent weeks; check errors on outbreak weeks separately
Leakage check	Is this week's rainfall known when forecasting? If not, use lagged rainfall only

Applying this in the assignments

Assessment	What to use from this week
Assignment 1 (due this week)	Regression option: two models plus a mean-predictor baseline, compared with cross-validation; report MAE, RMSE and R^2
Assignment 2 (due Week 7)	Time-series option: lag and rolling features, time-based split, TimeSeriesSplit tuning, naive baseline, MAE/RMSE/MAPE
All work	Check for leakage, and analyse residuals or errors, not just one score

Key takeaways

- Regression predicts numbers; most Week 4 classifiers have a regressor version
- Regularisation (Ridge, Lasso) controls overfitting in linear models
- Forecasting: build features from the past, split by time, and beat a naive baseline
- Report MAE or RMSE in target units, plus R^2 or MAPE; then study the residuals
- Watch for overfitting, imbalanced data and leakage; results that look too good usually are